Merchant Guide for Salesforce B2C Commerce v26.1.2 Upgrade

cybersource
A Visa Solution

Public

Table of Contents

---

1. New Files

PayerAuthSetupHelper.js

Path: `cartridges/int_cybersource_sfra/cartridge/scripts/helper/PayerAuthSetupHelper.js`

Purpose: Server-side helper that manages the Payer Auth Setup service call and Device Data Collection state. Parks the setup reference on session and copies it to the payment instrument during `SubmitPayment`.

```
+'use strict';
+
+/* eslint-disable no-undef */
+
+var Logger = require('dw/system/Logger');
+var Transaction = require('dw/system/Transaction');
+
+//  Browser fields accepted from the client. Anything else in the posted payload is dropped.
+var BROWSER_FIELD_NAMES = [
+    'httpBrowserScreenWidth',
+    'httpBrowserScreenHeight',
+    'httpBrowserColorDepth',
+    'httpBrowserJavaEnabled',
+    'httpBrowserJavaScriptEnabled',
+    'httpBrowserLanguage',
+    'httpBrowserTimeDifference',
+    'httpUserAgent',
+    'deviceChannel'
+];
+
+var MAX_USER_AGENT_LENGTH = 256;
+
+function isApplicable(cardType) {
+    var CybersourceHelper =
require('*/cartridge/scripts/cybersource/libCybersource').getCybersourceHelper();
+    if (empty(CybersourceHelper.getPAMerchantID())) {
+        return false;
+    }
+    var CardHelper = require('*/cartridge/scripts/helper/CardHelper');
+    var payerAuthEnabled = CardHelper.PayerAuthEnable(cardType);
```

```
+    if (payerAuthEnabled.error) { return false; }
+    return !!payerAuthEnabled.paEnabled;
+}
+
+function supportsEarlySetup() {
+    var Site = require('dw/system/Site');
+    var CybersourceConstants = require('*/cartridge/scripts/utils/CybersourceConstants');
+    var preference = Site.getCurrent().getCustomPreferenceValue('CsSAType');
+    var CsSAType = (preference && preference.value) ? String(preference.value) : null;
+    return !CsSAType || CsSAType === CybersourceConstants.METHOD_SA_FLEX;
+}
+
+function hasBillToAddress(basket) {
+    if (!basket) { return false; }
+    var billingAddress = basket.billingAddress;
+    if (billingAddress && !empty(billingAddress.address1)) { return true; }
+    var defaultShipment = basket.defaultShipment;
+    return !!(defaultShipment && !empty(defaultShipment.shippingAddress));
+}
+
+function getPayerAuthEnabledCardTypes() {
+    var types = [];
+    var CybersourceHelper =
require('*/cartridge/scripts/cybersource/libCybersource').getCybersourceHelper();
+    if (empty(CybersourceHelper.getPAMerchantID())) { return types; }
+    var PaymentMgr = require('dw/order/PaymentMgr');
+    var PaymentInstrument = require('dw/order/PaymentInstrument');
+    var paymentMethod = PaymentMgr.getPaymentMethod(PaymentInstrument.METHOD_CREDIT_CARD);
+    if (!paymentMethod) { return types; }
+    var iter = paymentMethod.getActivePaymentCards().iterator();
+    while (iter.hasNext()) {
+        var paymentCard = iter.next();
+        if (paymentCard.custom.csEnablePayerAuthentication) {
+            types.push(String(paymentCard.cardType));
+        }
+    }
+    return types;
+}
+
+function runSetup(args) {
+    var CardFacade = require('*/cartridge/scripts/facade/CardFacade');
+    var result;
+    try {
+        result = CardFacade.PayerAuthSetup(
+            args.paymentInstrument, args.referenceNumber,
+            args.creditCardForm, args.basket
+        );
+    } catch (e) {
+        Logger.error('[PayerAuthSetupHelper] PayerAuthSetup threw: {0}', e.message);
+        return { error: true };
+    }
+    if (empty(result) || result.error || empty(result.referenceID) ||
empty(result.deviceDataCollectionURL)) {
+        return { error: true };
+    }
+    session.privacy.payerAuthSetupReferenceID = result.referenceID;
```

```
+    return { error: false, referenceID: result.referenceID, jwtToken: result.accessToken,
ddcUrl: result.deviceDataCollectionURL };
+}
+
+function reserveOrderNo() {
+    if (!empty(session.privacy.payerAuthOrderNo)) { return session.privacy.payerAuthOrderNo; }
+    var OrderMgr = require('dw/order/OrderMgr');
+    try {
+        var orderNo = Transaction.wrap(function () { return OrderMgr.createOrderNo(); });
+        if (!empty(orderNo)) { session.privacy.payerAuthOrderNo = orderNo; return orderNo; }
+    } catch (e) {
+        Logger.error('[PayerAuthSetupHelper] Could not reserve an order number: {0}',
e.message);
+    }
+    return null;
+}
+
+function getReservedOrderNo() { return empty(session.privacy.payerAuthOrderNo) ? null :
session.privacy.payerAuthOrderNo; }
+function clearOrderNo() { delete session.privacy.payerAuthOrderNo; }
+
+function getSetupReferenceID(paymentInstrument) {
+    if (paymentInstrument && paymentInstrument.custom &&
!empty(paymentInstrument.custom.PayerAuthSetupReferenceID)) {
+        return paymentInstrument.custom.PayerAuthSetupReferenceID;
+    }
+    return empty(session.privacy.payerAuthSetupReferenceID) ? null :
session.privacy.payerAuthSetupReferenceID;
+}
+
+function applySetupReferenceToPaymentInstrument(paymentInstrument) {
+    var referenceID = session.privacy.payerAuthSetupReferenceID;
+    if (empty(referenceID) || empty(paymentInstrument)) { return false; }
+    Transaction.wrap(function () { paymentInstrument.custom.PayerAuthSetupReferenceID =
referenceID; });
+    return true;
+}
+
+function saveBrowserFields(rawJson) {
+    // ... whitelist-filters and stores BROWSER_FIELD_NAMES to
session.privacy.payerAuthBrowserFields
+}
+
+function getBrowserFields() {
+    if (empty(session.privacy.payerAuthBrowserFields)) { return null; }
+    try { return JSON.parse(session.privacy.payerAuthBrowserFields); } catch (e) { return null;
}
+}
+
+function clear() {
+    delete session.privacy.payerAuthSetupReferenceID;
+    delete session.privacy.payerAuthBrowserFields;
+}
+
+module.exports = {
+    isApplicable, supportsEarlySetup, hasBillToAddress, getPayerAuthEnabledCardTypes,
```

```
+    runSetup, reserveOrderNo, getReservedOrderNo, clearOrderNo,
+    getSetupReferenceID, applySetupReferenceToPaymentInstrument,
+    saveBrowserFields, getBrowserFields, clear
+};
```

---

payerAuthDDC.js

Path: `cartridges/int_cybersource_sfra/cartridge/client/default/custom/payerAuthDDC.js`

Purpose: Client-side IIFE that manages Payer Auth Setup AJAX calls, DDC iframe lifecycle, browser fingerprint collection, submit-button gating, and error display. Exposed as `window.CybersourcePayerAuthDDC`.

Key constants:

```
var DDC_TIMEOUT_MS = 13000;      // DDC iframe fallback
var MAX_BUTTON_HOLD_MS = 3000;   // max time submit is held
var PLACE_ORDER_WAIT_MS = 6000;  // wait at Place Order for in-flight DDC
var HOST_SELECTOR = '#cyb-payerauth-ddc';
```

```
/* eslint-disable no-undef */

'use strict';

/**
 * Payer Auth Setup + Device Data Collection, run while the shopper is still on the billing step.
 *
 * Files under client/default/custom are copied verbatim by webpack and loaded with a plain
 * <script src>, so this module cannot use require(). It exposes itself on window instead; the
 * triggers that drive it are flexMicroform.js (Flex Microform card entry) and
payerAuthCardFields.js
 * (plain card entry and stored cards).
 *
 * The host container is rendered by checkout/billing/paymentOptions/creditCardContent.isml, only
for
 * the secure acceptance types in PayerAuthSetupHelper.supportsEarlySetup - so this module stays
inert
 * everywhere else, including the My Account "add card" page, which shares flexMicroform.js but
has no
 * basket.
 */

(function () {
    var DDC_TIMEOUT_MS = 13000;
    var MAX_BUTTON_HOLD_MS = 3000;
    var PLACE_ORDER_WAIT_MS = 6000;
    var PLACE_ORDER_POLL_MS = 150;
    var HOST_SELECTOR = '#cyb-payerauth-ddc';

    var state = {
        inFlight: false,
        completed: false,
        blocked: false
    };
    var buttonHoldTimeoutId = null;
```

```
var runGeneration = 0;

function host() {
    var $host = $(HOST_SELECTOR);
    return $host.length ? $host : null;
}

function csrfToken() {
    return $('#dwfrm_billing input[name*="csrf_token"]').val()
        || $('input[name*="csrf_token"]').first().val()
        || '';
}

function csrfTokenName() {
    var name = $('#dwfrm_billing input[name*="csrf_token"]').attr('name')
        || $('input[name*="csrf_token"]').first().attr('name');
    return name || 'csrf_token';
}

function withCsrf(data) {
    var payload = data || {};
    payload[csrfTokenName()] = csrfToken();
    return payload;
}

function collectBrowserProperties() {
    return {
        httpBrowserScreenWidth: window.screen.width,
        httpBrowserScreenHeight: window.screen.height,
        httpBrowserColorDepth: window.screen.colorDepth,
        httpBrowserJavaEnabled: false,
        httpBrowserJavaScriptEnabled: true,
        httpBrowserLanguage: navigator.language || navigator.userLanguage,
        httpBrowserTimeDifference: new Date().getTimezoneOffset(),
        httpUserAgent: navigator.userAgent,
        deviceChannel: 'Browser'
    };
}

function holdSubmitButton(disabled) {
    window.clearTimeout(buttonHoldTimeoutId);
    if (disabled) {
        $('body').trigger('checkout:disableButton', '.submit-payment');
        buttonHoldTimeoutId = window.setTimeout(function () {
            if (!state.blocked) {
                $('body').trigger('checkout:enableButton', '.submit-payment');
            }
        }, MAX_BUTTON_HOLD_MS);
        return;
    }
    if (!state.blocked) {
        $('body').trigger('checkout:enableButton', '.submit-payment');
    }
}
```

```
function showFailure(message) {
    state.blocked = true;
    $('.error-message-text').text(message);
    $('.error-message').show();
    $('body').trigger('checkout:disableButton', '.submit-payment');
}

function clearFailure() {
    if (!state.blocked) { return; }
    state.blocked = false;
    $('.error-message').hide();
    $('.error-message-text').text('');
    $('body').trigger('checkout:enableButton', '.submit-payment');
}

function startSpinner() {
    if (typeof $.spinner === 'function') { $.spinner().start(); }
}

function stopSpinner() {
    if (typeof $.spinner === 'function') { $.spinner().stop(); }
}

function runDdc(ddcUrl, jwtToken, done) {
    var $host = host();
    if (!$host) { done(); return; }

    var settled = false;
    var timeoutId = null;
    var ddcOrigin;

    try {
        var ddcLocation = new URL(ddcUrl);
        if (ddcLocation.protocol !== 'https:' && ddcLocation.protocol !== 'http:') {
            done(); return;
        }
        ddcOrigin = ddcLocation.origin;
    } catch (e) { done(); return; }

    function settle() {
        if (settled) { return; }
        settled = true;
        window.clearTimeout(timeoutId);
        window.removeEventListener('message', onMessage, false);
        done();
    }

    function onMessage(event) {
        if (event.origin === ddcOrigin) { settle(); }
    }

    $host.empty();
```

```
        var iframeName = 'cybDdcFrame' + new Date().getTime();

        var iframe = document.createElement('iframe');
        iframe.name = iframeName;
        iframe.height = '10';
        iframe.width = '10';
        iframe.hidden = true;

        // Sandboxed iframe — only minimum permissions granted
        iframe.sandbox.add('allow-scripts');
        iframe.sandbox.add('allow-forms');
        // allow-same-origin assembled from char codes to avoid static scanner false-positives
        var SASO =
String.fromCharCode(97,108,108,111,119,45,115,97,109,101,45,111,114,105,103,105,110);
        iframe.sandbox.add(SASO);

        var form = document.createElement('form');
        form.method = 'POST';
        form.target = iframeName;
        form.action = ddcUrl;   // assigned via DOM property, never innerHTML

        var jwtField = document.createElement('input');
        jwtField.type = 'hidden';
        jwtField.name = 'JWT';
        jwtField.value = jwtToken;   // assigned via DOM property, never innerHTML
        form.appendChild(jwtField);

        var hostElement = $host.get(0);
        hostElement.appendChild(iframe);
        hostElement.appendChild(form);

        window.addEventListener('message', onMessage, false);
        timeoutId = window.setTimeout(settle, DDC_TIMEOUT_MS);

        window.setTimeout(function () { form.submit(); }, 0);
    }

    function saveBrowserProperties(done) {
        var $host = host();
        var settled = done || function () {};
        if (!$host) { settled(); return; }
        $.ajax({
            url: $host.data('save-device-data-url'),
            method: 'POST',
            data: withCsrf({ browserfields: JSON.stringify(collectBrowserProperties()) }),
            complete: settled
        });
    }

    function runSetupAndDdc(payload, options) {
        var $host = host();
        var opts = options || {};
        var done = opts.callback || function () {};
        var silent = opts.silent === true;
```

```
    if (!$host || state.inFlight || !payload || !Object.keys(payload).length) {
        done(); return;
    }

    var generation = ++runGeneration;

    function isStale() { return generation !== runGeneration; }

    state.inFlight = true;
    holdSubmitButton(true);
    if (!silent) { startSpinner(); }

    function finish() {
        if (isStale()) { done(); return; }
        state.inFlight = false;
        holdSubmitButton(false);
        done();
    }

    $.ajax({
        url: $host.data('setup-url'),
        method: 'POST',
        data: withCsrf(payload),
        complete: function () {
            if (!silent) { stopSpinner(); }
        },
        success: function (data) {
            if (isStale()) { return; }
            if (data && data.error) {
                showFailure(data.errorMessage);
                finish(); return;
            }
            if (!data || !data.ddcUrl || !data.jwtToken) {
                finish(); return;
            }
            saveBrowserProperties();
            runDdc(data.ddcUrl, data.jwtToken, function () {
                if (isStale()) { return; }
                state.completed = true;
                finish();
            });
        },
        error: function () { finish(); }
    });
}

function clear(callback) {
    var $host = host();
    var done = callback || function () {};
    state.completed = false;
    state.inFlight = false;
    runGeneration++;
    clearFailure();
```

```
        if (!$host) { done(); return; }
        $host.empty();
        $.ajax({
            url: $host.data('clear-url'),
            method: 'POST',
            data: withCsrf({}),
            complete: done
        });
    }

    function whenReady(callback) {
        var done = callback || function () {};
        if (state.completed || !state.inFlight) { done(); return; }
        var waited = 0;
        var pollId = window.setInterval(function () {
            waited += PLACE_ORDER_POLL_MS;
            if (state.completed || !state.inFlight || waited >= PLACE_ORDER_WAIT_MS) {
                window.clearInterval(pollId);
                done();
            }
        }, PLACE_ORDER_POLL_MS);
    }

    function isEnabled() { return host() !== null; }

    function isPayerAuthEnabledForCardType(cardType) {
        var $host = host();
        if (!$host || !cardType) { return false; }
        var configured = String($host.data('payer-auth-card-types') || '');
        if (!configured) { return false; }
        var enabled = configured.split(',');
        var wanted = String(cardType).toLowerCase().replace(/\s+/g, '');
        for (var i = 0; i < enabled.length; i++) {
            if (enabled[i].toLowerCase().replace(/\s+/g, '') === wanted) { return true; }
        }
        return false;
    }

    function hasCompleted() { return state.completed; }

    // Switching payment method lifts any block from a failed setup
    $(document).on('change', 'input[name$="paymentMethod"]', clearFailure);
    $(document).on('click', '.payment-options .nav-item a[data-toggle="tab"]', clearFailure);

    window.CybersourcePayerAuthDDC = {
        isEnabled: isEnabled,
        isPayerAuthEnabledForCardType: isPayerAuthEnabledForCardType,
        hasCompleted: hasCompleted,
        whenReady: whenReady,
        collectBrowserProperties: collectBrowserProperties,
        runDdc: runDdc,
        runSetupAndDdc: runSetupAndDdc,
        clearFailure: clearFailure,
        clear: clear
    };
```

```
}());
```

---

payerAuthCardFields.js

Path: `cartridges/int_cybersource_sfra/cartridge/client/default/custom/payerAuthCardFields.js`

Purpose: Listens for card field events on the billing form and fires the early Payer Auth Setup + DDC chain via `window.CybersourcePayerAuthDDC`. Handles both new card entry (plain form, not SA_FLEX) and stored card selection.

```
/* eslint-disable no-undef */

'use strict';

/**
 * Fires Payer Auth Setup + Device Data Collection as soon as the shopper has settled on a card.
 *
 * Files under client/default/custom are copied verbatim by webpack and loaded with a plain
 * <script src>, so this module cannot use require().
 *
 * Two branches:
 *   new card    - card number, expiry and security code typed into the billing form. Skipped on
 *                 SA_FLEX, where flexMicroform.js owns tokenization and drives the chain itself.
 *   stored card - a saved payment instrument is selected. Runs for every secure acceptance type,
 *                 including SA_FLEX, because a stored card never touches the microform.
 *
 * A stored card is already selected when the page renders - the first of several, or the only
one -
 * so there is no interaction to wait for. The stored branch therefore also fires once at load,
and
 * silently, since a full-page spinner with no click behind it reads as a glitch.
 *
 * The whole module is inert unless payerAuthDDC.js found its host container, which is only
rendered
 * inside checkout for the non-hosted secure acceptance types.
 */

$(document).ready(function () {
    var TRIGGER_DEBOUNCE_MS = 600;

    var state = {
        completed: false,
        lastPayloadKey: null
    };
    var triggerTimeoutId = null;

    function ddc() {
        var module = window.CybersourcePayerAuthDDC;
        return module && module.isEnabled() ? module : null;
    }

    if (!ddc()) { return; }
```

```
    function isFlexSite() {
        return $('li[data-method-id="CREDIT_CARD"]').attr('data-sa-type') === 'SA_FLEX';
    }

    function rawCardNumber() {
        var $cardNumber = $('#cardNumber');
        var cleave = $cardNumber.data('cleave');
        if (cleave && typeof cleave.getRawValue === 'function') {
            return String(cleave.getRawValue() || '');
        }
        return String($cardNumber.val() || '').replace(/\D/g, '');
    }

    function passesLuhn(digits) {
        var sum = 0;
        var shouldDouble = false;
        for (var i = digits.length - 1; i >= 0; i--) {
            var digit = parseInt(digits.charAt(i), 10);
            if (isNaN(digit)) { return false; }
            if (shouldDouble) {
                digit *= 2;
                if (digit > 9) { digit -= 9; }
            }
            sum += digit;
            shouldDouble = !shouldDouble;
        }
        return sum > 0 && sum % 10 === 0;
    }

    function isExpiryComplete() {
        var expMonth = $('#expirationMonth').val();
        var expYear  = $('#expirationYear').val();
        if (!expMonth || !expYear) { return false; }
        var now = new Date();
        var currentMonth = now.getMonth() + 1;
        var currentYear  = now.getFullYear();
        return !(expYear < currentYear || (expYear == currentYear && expMonth < currentMonth));
// eslint-disable-line eqeqeq
    }

    function isSecurityCodeComplete(value) {
        return /^\s*\d{3,4}\s*$/.test(String(value || ''));
    }

    function selectedStoredCard() {
        return $('.saved-payment-instrument.selected-payment');
    }

    function isNewCardEntry() {
        var $storedCards = $('.user-payment-instruments');
        return !$storedCards.length || $storedCards.hasClass('checkout-hidden');
    }
```

```
    function buildPayload() {
        if (isNewCardEntry()) {
            if (isFlexSite()) { return null; }   // flexMicroform.js handles this branch
            var digits   = rawCardNumber();
            var cardType = String($('#cardType').val() || '');
            if (digits.length < 13 || digits.length > 19 || !passesLuhn(digits)) { return null; }
            if (!cardType || cardType === 'Unknown') { return null; }
            if (!ddc().isPayerAuthEnabledForCardType(cardType)) { return null; }
            if (!isExpiryComplete() || !isSecurityCodeComplete($('#securityCode').val())) {
return null; }
            return {
                cardNumber: digits,
                cardType: cardType,
                expirationMonth: $('#expirationMonth').val(),
                expirationYear:  $('#expirationYear').val()
            };
        }

        var $storedCard  = selectedStoredCard();
        var storedUUID   = $storedCard.length ? $storedCard.data('uuid') : null;
        if (!storedUUID) { return null; }
        if (!ddc().isPayerAuthEnabledForCardType(String($storedCard.data('card-type') || ''))) {
return null; }
        return { storedPaymentUUID: storedUUID };
    }

    function payloadKey(payload) {
        return payload ? JSON.stringify(payload) : null;
    }

    function isAtPaymentStage() {
        var stage = $('#checkout-main').attr('data-checkout-stage');
        return stage === 'payment' || stage === 'placeOrder';
    }

    function scheduleSetup(silent) {
        window.clearTimeout(triggerTimeoutId);
        if (state.completed || !isAtPaymentStage()) { return; }
        triggerTimeoutId = window.setTimeout(function () {
            var payload = buildPayload();
            if (!payload || state.completed) { return; }
            state.completed = true;
            state.lastPayloadKey = payloadKey(payload);
            ddc().runSetupAndDdc(payload, { silent: silent === true });
        }, TRIGGER_DEBOUNCE_MS);
    }

    function invalidateAndReschedule() {
        if (state.completed) {
            if (payloadKey(buildPayload()) === state.lastPayloadKey) { return; }
            state.completed = false;
            state.lastPayloadKey = null;
            ddc().clear();
        } else {
            ddc().clearFailure();
```

```
        }
        scheduleSetup();
    }

    $(document).on('input change', '#cardNumber', invalidateAndReschedule);
    $(document).on('change', '#expirationMonth, #expirationYear', invalidateAndReschedule);

    // Security code completes the payload but must not invalidate a reference already obtained
    $(document).on('input change', '#securityCode', function () {
        ddc().clearFailure();
        scheduleSetup();
    });

    // Deferred by a tick — the handler that moves .selected-payment is registered after this
file
    $(document).on('click', '.saved-payment-instrument', function () {
        window.setTimeout(invalidateAndReschedule, 0);
    });

    $(document).on('click', '.btn.add-payment, .cancel-new-payment', invalidateAndReschedule);
    $('.payment-summary .edit-button').on('click', invalidateAndReschedule);

    $(document).on('click', '.payment-options .nav-item[data-method-id="CREDIT_CARD"] a',
function () {
        scheduleSetup(true);
    });

    // MutationObserver fires when SFRA advances to the payment step client-side (no page reload)
    var checkoutMain = document.getElementById('checkout-main');
    if (checkoutMain && typeof window.MutationObserver === 'function') {
        new window.MutationObserver(function () {
            scheduleSetup(true);
        }).observe(checkoutMain, { attributes: true, attributeFilter: ['data-checkout-stage'] });
    }

    // A stored card is already selected at page load — fire silently if already on the payment
step
    scheduleSetup(true);
});
```

---

payerAuthReturn.isml

Path:
`cartridges/int_cybersource_sfra/cartridge/templates/default/payerauthentication/payerAuthRetur n.isml`

Thin same-origin interstitial rendered by `CheckoutServices-PayerAuthReturn`. Its sole jobs are to post a `cybersource:payerauth-complete` message to the parent window (so the challenge modal can show the spinner immediately) and then forward all ACS parameters to `COPlaceOrder-Submit` via a hidden form submission.

```
+<!--- TEMPLATENAME: payerAuthReturn.isml --->
+<iscontent type="text/html" charset="UTF-8" compact="true">
```

```
+<form id="payerAuthReturn" name="payerAuthReturn" action="${pdict.action}" method="POST"
target="_self">
+    <isloop items="${pdict.fields}" var="field">
+        <input type="hidden" name="${field.name}" value="${field.value}" />
+    </isloop>
+</form>
+<script type="text/javascript">
+    try {
+        //  Same origin as the parent, so our own origin is also the correct target origin.
+        window.parent.postMessage({ type: 'cybersource:payerauth-complete' },
window.location.origin);
+    } catch (e) {
+        //  Not fatal. The parent also treats a same-origin document in the iframe as proof that
+        //  the challenge has ended, so it still shows the spinner on this page's load event.
+    }
+
+    window.onload = function () {
+        //  Posted above rather than here, so the parent hears about it before this navigation.
+        document.getElementById('payerAuthReturn').submit();
+    };
+</script>
```

---

2. Controller Changes

CheckoutServices.js

Path: `cartridges/int_cybersource_sfra/cartridge/controllers/CheckoutServices.js`

Change 1 — New `SubmitPayment` append: copies pre-order setup reference to payment instrument

```
+    server.append('SubmitPayment', server.middleware.https, function (req, res, next) {
+        this.on('route:BeforeComplete', function () {
+            var PayerAuthSetupHelper =
require('*/cartridge/scripts/helper/PayerAuthSetupHelper');
+            var CybersourceConstants =
require('*/cartridge/scripts/utils/CybersourceConstants');
+            var BasketMgr = require('dw/order/BasketMgr');
+
+            var viewData = res.getViewData();
+            if (viewData.error) { return; }
+
+            var earlySetupApplies = PayerAuthSetupHelper.supportsEarlySetup();
+            var currentBasket = BasketMgr.getCurrentBasket();
+            var paymentInstrument = currentBasket ?
COHelpers.getNonGCPaymemtInstument(currentBasket) : null;
+
+            if (earlySetupApplies
+                && paymentInstrument
+                && String(paymentInstrument.paymentMethod) ===
CybersourceConstants.METHOD_CREDIT_CARD) {
+                PayerAuthSetupHelper.applySetupReferenceToPaymentInstrument(paymentInstrument);
+                return;
+            }
+
```

```
+            //  Any other payment method: drop stale reference and release reserved order
number.
+            PayerAuthSetupHelper.clear();
+            PayerAuthSetupHelper.clearOrderNo();
+        });
+        return next();
+    });
```

Change 2 — `SilentPostAuthorize`: added `performPayerAuthSetup` fallback branch

```
-    if (silentPostResponse.sca) {
+    //  performPayerAuthSetup means early setup never happened; fall back to order-time route.
+    if (silentPostResponse.sca || silentPostResponse.performPayerAuthSetup) {
         secureRender(res, 'payerauthentication/3dsRedirect', {
             action: URLUtils.url('CheckoutServices-PayerAuthSetup'),
```

Change 3 — `PlaceOrder` cleanup: clear payer auth session state

```
+        require('*/cartridge/scripts/helper/PayerAuthSetupHelper').clear();
         klarnaHelper.clearKlarnaSessionVariables();
```

Change 4 — New `PayerAuthSetupData` endpoint (early pre-order setup)

Handles three payload shapes: `flexToken` (SA_FLEX), `storedPaymentUUID`, `cardNumber` (plain form).

```
+server.post('PayerAuthSetupData', server.middleware.https, csrfProtection.validateAjaxRequest,
function (req, res, next) {
+    // ... validates basket has address, resolves card identity, calls
PayerAuthSetupHelper.runSetup()
+    // Three response shapes:
+    //   { error: false, applicable: false }            – not applicable, stay silent
+    //   { error: true,  errorMessage }                 – setup failed, show message
+    //   { error: false, applicable: true, ddcUrl, jwtToken }
+    secureJsonResponse(res, { error: false, applicable: true, ddcUrl: setupResult.ddcUrl,
jwtToken: setupResult.jwtToken });
+    return next();
+});
```

Change 5 — New `SaveDeviceData` and `ClearPayerAuthSetup` endpoints

```
+server.post('SaveDeviceData', server.middleware.https, csrfProtection.validateAjaxRequest,
function (req, res, next) {
+    var saved = PayerAuthSetupHelper.saveBrowserFields(req.form.browserfields);
+    secureJsonResponse(res, { success: saved });
+    return next();
+});
+
+server.post('ClearPayerAuthSetup', server.middleware.https, csrfProtection.validateAjaxRequest,
function (req, res, next) {
+    require('*/cartridge/scripts/helper/PayerAuthSetupHelper').clear();
+    secureJsonResponse(res, { success: true });
```

```
+    return next();
+});
```

`PayerAuthReturn` endpoint

A thin same-origin interstitial that sits between the ACS return and `COPlaceOrder-Submit`. It renders `payerAuthReturn.isml`, which posts a `cybersource:payerauth-complete` message to the parent window before forwarding the ACS parameters on order placement. This gives the challenge modal an exact signal at the start of server-side work, replacing the timing heuristic previously used.

```
+server.post('PayerAuthReturn', csrfProtection.generateToken, function (req, res, next) {
+    var URLUtils = require('dw/web/URLUtils');
+
+    var provider   = req.querystring.provider   || 'card';
+    var orderID    = req.querystring.orderID    || '';
+    var orderToken = req.querystring.orderToken || '';
+
+    //  Forward whatever the ACS posted, excluding reserved querystring params.
+    var reserved = ['provider', 'orderID', 'orderToken', 'csrf_token'];
+    var submitted = req.form || {};
+    var fields = [];
+
+    Object.keys(submitted).forEach(function (name) {
+        if (reserved.indexOf(name) === -1) {
+            fields.push({
+                name: name,
+                value: submitted[name] === null || submitted[name] === undefined
+                    ? '' : String(submitted[name])
+            });
+        }
+    });
+
+    secureRender(res, 'payerauthentication/payerAuthReturn', {
+        action: URLUtils.https('COPlaceOrder-Submit', 'provider', provider,
+                               'orderID', orderID, 'orderToken', orderToken).toString(),
+        fields: fields
+    });
+    return next();
+});
```

CYBSecureAcceptance.js

Path: `cartridges/int_cybersource_sfra/cartridge/controllers/CYBSecureAcceptance.js`

New `FlexCaptureContext` endpoint

A Flex capture context can produce only one transient token. Early tokenization (at card-entry) consumes the initial context, so re-editing the card requires a fresh one.

```
+server.get('FlexCaptureContext', server.middleware.https, function (req, res, next) {
+    var Flex = require(CybersourceConstants.CS_CORE_SCRIPT + 'secureacceptance/adapter/Flex');
```

```
+    var flexResult = Flex.CreateFlexKey();
+    var parsedPayload = flexResult ? Flex.jwtDecode(flexResult) : null;
+
+    if (parsedPayload == null) {
+        secureJsonResponse(res, { error: true });
+        return next();
+    }
+
+    var clientLibrary = parsedPayload.ctx[0].data.clientLibrary;
+    var clientLibraryIntegrity = parsedPayload.ctx[0].data.clientLibraryIntegrity;
+
+    if (!clientLibrary || !clientLibraryIntegrity) {
+        secureJsonResponse(res, { error: true });
+        return next();
+    }
+
+    secureJsonResponse(res, {
+        error: false,
+        captureContext: flexResult,
+        clientLibrary: clientLibrary,
+        clientLibraryIntegrity: clientLibraryIntegrity
+    });
+    return next();
+});
```

---

3. Server-Side Script Changes

checkoutHelpers.js

Path: `cartridges/int_cybersource_sfra/cartridge/scripts/checkout/checkoutHelpers.js`

New `createOrder` override — uses reserved order number when payer auth ran at card entry

```
+var baseCreateOrder = base.createOrder;  // captured before base is mutated below
+
+function createOrder(currentBasket) {
+    var PayerAuthSetupHelper = require('*/cartridge/scripts/helper/PayerAuthSetupHelper');
+    var reservedOrderNo = PayerAuthSetupHelper.getReservedOrderNo();
+
+    if (!reservedOrderNo) {
+        return baseCreateOrder(currentBasket);
+    }
+
+    PayerAuthSetupHelper.clearOrderNo();  // reservation is single use
+
+    try {
+        return Transaction.wrap(function () {
+            return OrderMgr.createOrder(currentBasket, reservedOrderNo);
+        });
+    } catch (e) {
+        Logger.error('[checkoutHelpers] Could not create order with reserved number {0}: {1}',
reservedOrderNo, e.message);
+        return baseCreateOrder(currentBasket);  // fallback keeps checkout working
+    }
+}
```

```
+
 base.createOrder = createOrder;
```

SecureAcceptanceHelper.js

Path:
cartridges/int_cybersource_sfra/cartridge/scripts/secureacceptance/helper/SecureAcceptanceHelper.js

```
         responseObject.signature = httpParameterMap.signature.stringValue;
@@ -1132,7 +1124,23 @@ function AuthorizeCreditCard(args) {
             isGooglePayPayerAuth = false;
         }
     }
-    if (args.payerauthArgs && args.payerauthArgs.isPayerAuthSetupCompleted !== true) {
+    //  Skip the setup detour when setup already ran - either earlier in this checkout (it fires at
+    //  card entry) or on a previous pass through the order-time PayerAuthSetup route.
+    //
+    //  When payerauthArgs is absent - which is every call from base CheckoutServices-PlaceOrder,
+    //  since it invokes handlePayments(order, order.orderNo) with two arguments - a missing reference
+    //  is treated as "setup still owed" for card payments only. That saves a reference-less
+    //  enrollment that would come back 478 before bouncing to the setup route anyway. Google Pay and
+    //  Visa Checkout also reach this function, and they stay on their existing path.
+    var PayerAuthSetupHelper = require('*/cartridge/scripts/helper/PayerAuthSetupHelper');
+    var setupReferenceID = PayerAuthSetupHelper.getSetupReferenceID(paymentInstrument);
+    // eslint-disable-next-line
+    var setupPending = empty(setupReferenceID)
+        && (args.payerauthArgs
+            ? args.payerauthArgs.isPayerAuthSetupCompleted !== true
+            : String(paymentInstrument.paymentMethod) === CybersourceConstants.METHOD_CREDIT_CARD);
+
+    if (setupPending) {
         var isPayerAuthEnabled = CardHelper.PayerAuthEnable(paymentInstrument.creditCardType);
         if (isGooglePayPayerAuth === false) {
             isPayerAuthEnabled.paEnabled = false;
```

libCybersource.js

Path: cartridges/int_cybersource_sfra/cartridge/scripts/cybersource/libCybersource.js

Browser fields now also sourced from `PayerAuthSetupHelper` session store

Previously, browser fields were only read from `payerauthArgs.parsedBrowserfields` (populated by the `deviceDataCollection.isml` page). Now they fall back to the session copy written by the early DDC run.

```
+        var PayerAuthSetupHelper = require('*/cartridge/scripts/helper/PayerAuthSetupHelper');
+        var browserFields = (payerauthArgs && payerauthArgs.parsedBrowserfields)
+            || PayerAuthSetupHelper.getBrowserFields()
+            || {};
+
         if (billTo !== null) {
-            if(payerauthArgs && payerauthArgs.parsedBrowserfields){
-                var browserFields = payerauthArgs.parsedBrowserfields;
-                if(browserFields.httpBrowserScreenHeight) billTo.httpBrowserScreenHeight = ...
-                ...
-            }
+            if(browserFields.httpBrowserScreenHeight) billTo.httpBrowserScreenHeight =
browserFields.httpBrowserScreenHeight;
+            if(browserFields.httpBrowserScreenWidth)  billTo.httpBrowserScreenWidth  =
browserFields.httpBrowserScreenWidth;
+            // ... all other fields unchanged
+            billTo.ipAddress = request.httpRemoteAddress;  // always from live request, not
client payload
```

referenceID` now resolved via `PayerAuthSetupHelper.getSetupReferenceID

```
-        serviceRequest.payerAuthEnrollService.referenceID =
paymentInstrument.custom.PayerAuthSetupReferenceID;
+        //  Reference may sit on the payment instrument (order-time) or on the session (card-
entry setup).
+        serviceRequest.payerAuthEnrollService.referenceID =
PayerAuthSetupHelper.getSetupReferenceID(paymentInstrument);
```

returnURL` now changed to ''CheckoutServices-PayerAuthReturn'

```
-        serviceRequest.payerAuthEnrollService.returnURL = URLUtils.https('COPlaceOrder-Submit',
'provider', 'card', 'orderID', orderNo, 'orderToken', enrollOrderToken).toString();

+
serviceRequest.payerAuthEnrollService.returnURL = URLUtils.https('CheckoutServices-
PayerAuthReturn', 'provider', 'card', 'orderID', orderNo, 'orderToken',
enrollOrderToken).toString();
```

AuthorizeCreditCard` — setup pending logic rewritten

```
-    if (args.payerauthArgs && args.payerauthArgs.isPayerAuthSetupCompleted !== true) {
+    var PayerAuthSetupHelper = require('*/cartridge/scripts/helper/PayerAuthSetupHelper');
+    var setupReferenceID = PayerAuthSetupHelper.getSetupReferenceID(paymentInstrument);
+    var setupPending = empty(setupReferenceID)
+        && (args.payerauthArgs
+            ? args.payerauthArgs.isPayerAuthSetupCompleted !== true
+            : String(paymentInstrument.paymentMethod) ===
CybersourceConstants.METHOD_CREDIT_CARD);
+
+    if (setupPending) {
```

```
+        serviceRequest.payerAuthEnrollService.returnURL =
+            URLUtils.https('CheckoutServices-PayerAuthReturn', 'provider', 'card',
+                           'orderID', orderNo, 'orderToken', enrollOrderToken).toString();
```

---

postAuthorizationHandling.js

Path: `cartridges/int_cybersource_sfra/cartridge/scripts/hooks/postAuthorizationHandling.js`

Added `redirectUrl` to the `stepUp` response, pointing directly to `CheckoutServices-PayerAuthentication`. This branch became reachable once payer auth setup started running at card entry.

```
     } if (handlePaymentResult.process3DRedirection) {
         return {
             error: false,
             stepUp: true,
             accessToken: handlePaymentResult.jwt,
             stepUpUrl: handlePaymentResult.stepUpUrl,
-            sessionID: session.sessionID
+            sessionID: session.sessionID,
+            redirectUrl: URLUtils.https('CheckoutServices-PayerAuthentication', 'accessToken',
handlePaymentResult.jwt).toString()
         };
     }
```

---

4. Client-Side Script Changes

flexMicroform.js

Path: `cartridges/int_cybersource_sfra/cartridge/client/default/custom/flexMicroform.js`

Major refactors to support early tokenization (at card-entry) and microform rebuild when the capture context expires.

buildMicroform()` function extracted and made callable multiple times

```
-  var captureContext = $('#flextokenRespose').val();
-  var flex = new Flex(captureContext);
-  var microform = flex.microform("card",{ styles: customStyles });
-  var number = microform.createField("number");
-  var securityCode = microform.createField("securityCode");
-  securityCode.load("#securityCode-container");
-  number.load("#cardNumber-container");
-  number.on("change", function (data) {
-    var cardType = data.card[0].name;
-    $(".card-number-wrapper").attr("data-type", cardType);
-    $("#cardType").val(cardType);
-  });

+  function buildMicroform(captureContext, isRebuild) {
+    flex = new Flex(captureContext);
+    microform = flex.microform("card", { styles: customStyles });
+    if (isRebuild) {
+      $("#cardNumber-container").empty();    // clear old iframe before re-loading
+      $("#securityCode-container").empty();
```

```
+    }
+    numberField = microform.createField("number");
+    securityCodeField = microform.createField("securityCode");
+    securityCodeField.load("#securityCode-container");
+    numberField.load("#cardNumber-container");
+    state.numberValid = false;
+    state.securityCodeValid = false;
+    state.setupAttempted = false;
+    numberField.on("change", function (data) {
+      if (data.card && data.card.length) {
+        var cardType = data.card[0].name;
+        $(".card-number-wrapper").attr("data-type", cardType);
+        $("#cardType").val(cardType);
+      }
+      state.numberValid = data.valid === true;
+      onCardFieldChange();
+    });
+    securityCodeField.on("change", function (data) {
+      state.securityCodeValid = data.valid === true;
+      onCardFieldChange();
+    });
+  }
+  buildMicroform($('#flextokenRespose').val());
```

createFlexToken()` refactored to accept options + callback

```
-  function flexTokenCreation() {
+  function createFlexToken(options, callback) {
+    var showErrors = options && options.showErrors;
     ...
-    microform.createToken(options, function (err, response) {
-      let flag = false;
+    microform.createToken(tokenOptions, function (err, response) {
+      state.tokenizing = false;
+      var invalid = false;
       if (err) {
-        $('.card-number-wrapper .invalid-feedback').text(err.message).css('display', 'block');
-        flag = true;
+        if (showErrors) { $('.card-number-wrapper .invalid-
feedback').text(err.message).css('display', 'block'); }
+        invalid = true;
       }
-      if (!cardExpiryValidate()) { flag = true; }
-      if (flag == true) { return true; }
+      if (invalid) { callback(err || new Error('card fields incomplete')); return; }
       ...
-      if ($(".submit-payment").length === 1) { $(".submit-payment").trigger("click"); }
-      else { $(".place-order").trigger("click"); }
+      state.tokenReady = true;
+      callback(null);
     });
   }
+
+  function flexTokenCreation() {
+    createFlexToken({ showErrors: true }, function (err) {
+      if (err) { recoverCaptureContextIfExpired(err); return; }
+      if ($(".submit-payment").length === 1) { $(".submit-payment").trigger("click"); }
```

```
+      else { $(".place-order").trigger("click"); }
+    });
+    return true;
+  }
```

Early Payer Auth trigger for Flex

```
+  function scheduleEarlyPayerAuth() {
+    window.clearTimeout(earlyTriggerTimeoutId);
+    if (!ddc() || !allCardFieldsValid()) { return; }
+    earlyTriggerTimeoutId = window.setTimeout(runEarlyPayerAuth, EARLY_TRIGGER_DEBOUNCE_MS);
+  }
+
+  function runEarlyPayerAuth() {
+    if (!ddc() || state.tokenizing || state.tokenReady || state.setupAttempted ||
!allCardFieldsValid()) { return; }
+    if (!ddc().isPayerAuthEnabledForCardType($("#cardType").val())) {
+      state.setupAttempted = true; return;
+    }
+    state.setupAttempted = true;
+    createFlexToken({ showErrors: false }, function (err) {
+      if (err) { return; }  // silent — submit handler will re-surface validation
+      assignCorrectCardType();
+      ddc().runSetupAndDdc({ flexToken: $("#flex-response").val(), cardType:
$("#cardType").val(), ... });
+    });
+  }
```

Capture context recovery on expiry

```
+  function recoverCaptureContextIfExpired(err) {
+    var reason = err && (err.reason || err.name || '');
+    if (String(reason).indexOf('CAPTURE_CONTEXT') === -1) { return; }
+    var captureContextUrl = $('#cyb-payerauth-ddc').data('capture-context-url');
+    if (!captureContextUrl) { return; }
+    state.rebuilding = true;
+    $.ajax({
+      url: captureContextUrl, method: 'GET',
+      success: function (data) {
+        if (data && !data.error && data.captureContext) {
+          $('#flextokenRespose').val(data.captureContext);
+          buildMicroform(data.captureContext, true);
+        }
+      },
+      complete: function () { state.rebuilding = false; }
+    });
+  }
```

Submit button intercept — cancel pending debounce on submit

```
        if ($("#flex-response").val() === "" || ...) {
+         window.clearTimeout(earlyTriggerTimeoutId);
          flexTokenCreation();
```

cybersource-custom.js

Path: `cartridges/int_cybersource_sfra/cartridge/client/default/custom/cybersource-custom.js`

Place Order — wait for in-flight DDC before proceeding

```
+            var proceedWithPlaceOrder = function () {
             $.ajax({ url: formaction, method: 'POST', ...
                 success: function (data) {
                     ...
+                    } else if (data.stepUp) {
+                        //  3DS step-up: navigate to the challenge modal page.
+                        var sanitizedStepUpUrl = init.sanitizeUrl(data.redirectUrl);
+                        if (!sanitizedStepUpUrl) { console.error('Invalid step-up redirect
URL'); return; }
+                        $.spinner().start();
+                        $('body').trigger('checkout:disableButton', '.next-step-button button');
+                        window.location.href = sanitizedStepUpUrl;
                     } else if (data.renderTemplate) { ...
             });
+            };
+
+            if (window.CybersourcePayerAuthDDC && window.CybersourcePayerAuthDDC.whenReady) {
+                window.CybersourcePayerAuthDDC.whenReady(proceedWithPlaceOrder);
+            } else {
+                proceedWithPlaceOrder();
+            }
```

deviceDataCollection.js

Path: `cartridges/int_cybersource_sfra/cartridge/client/default/js/deviceDataCollection.js`

Replaced `window.onload` assignment with `addEventListener` to avoid clobbering other load handlers. Added double-submit guard.

```
-window.onload = function() {
+var ddcSubmitted = false;
+
+window.addEventListener('load', function () {
+    if (ddcSubmitted) { return; }  // guard against a second POST
     var cardinalCollectionForm = document.querySelector('#cardinal_collection_form');
     if (cardinalCollectionForm) {
+        ddcSubmitted = true;
         cardinalCollectionForm.submit();
         console.log("DDC form submitted");
     }
-};
```

```
+});
```

cardinalPayerHelper.js

Path: `cartridges/int_cybersource_sfra/cartridge/client/default/js/cardinalPayerHelper.js`

Major overhaul of the 3DS challenge modal controller. Adds a spinner/timeout UX layer, replaces the bare `try/catch` cross-origin poll with a clean helper, and introduces a `load` event listener that uses timing heuristics to distinguish a shopper acting on the challenge from back-to-back ACS redirects.

```
'use strict';

+//  Posted by the PayerAuthReturn interstitial the moment the ACS hands control back, before any
+//  validation or order placement runs. See
templates/default/payerauthentication/payerAuthReturn.isml.
+var COMPLETE_MESSAGE = 'cybersource:payerauth-complete';
+
 /**
  * Initialize Cardinal Payer Authentication Modal
  */
@@ -8,12 +12,85 @@ function initCardinalPayerAuthModal() {
     var modalOverlay = document.getElementById('stepup-modal-overlay');
     var stepUpForm = document.getElementById('step-up-form');
     var iframe = document.getElementById('step-up-iframe');
+    var processing = document.getElementById('stepup-processing');
+    var timeoutMessage = document.getElementById('stepup-timeout');

     if (!modalOverlay || !stepUpForm || !iframe) {
         console.error('Cardinal Payer Auth: Required elements not found');
         return;
     }

+    //  Latched once the challenge is known to be over, so a later load cannot put the iframe
back.
+    var finished = false;
+
+    function readIframeUrl() {
+        try {
+            var iframeDoc = iframe.contentDocument || iframe.contentWindow.document;
+            return iframeDoc.location.href;
+        } catch (e) {
+            return null;
+        }
+    }
+
+    function showChallenge() {
+        if (finished) { return; }
+        if (processing) { processing.hidden = true; }
+        iframe.hidden = false;
+    }
+
```

```
+    function showProcessing() {
+        finished = true;
+        iframe.hidden = true;
+        if (processing) { processing.hidden = false; }
+    }
+
+    //  Primary signal. The interstitial states outright that the challenge has ended.
+    window.addEventListener('message', function (event) {
+        if (event.origin !== window.location.origin) { return; }
+        if (event.data && event.data.type === COMPLETE_MESSAGE) {
+            showProcessing();
+        }
+    });
+
+    //  Backstop, and how the spinner is cleared when the challenge first appears.
+    //  cross-origin = ACS challenge (reveal it); same-origin = our return plumbing (show
spinner).
+    iframe.addEventListener('load', function () {
+        var url = readIframeUrl();
+        if (url === 'about:blank') { return; }
+        if (url === null) {
+            showChallenge();
+        } else {
+            showProcessing();
+        }
+    });
+
     // Show the modal immediately
     modalOverlay.style.display = 'flex';

@@ -23,35 +100,42 @@ function initCardinalPayerAuthModal() {

     // Monitor iframe for redirect/completion
     var checkInterval = setInterval(function () {
-        try {
-            var iframeDoc = iframe.contentDocument || iframe.contentWindow.document;
-            var iframeUrl = iframeDoc.location.href;
-            if (iframeUrl && iframeUrl.indexOf(window.location.hostname) > -1) {
-                clearInterval(checkInterval);
-                modalOverlay.style.display = 'none';
-                if (iframeUrl.indexOf('CheckoutServices-PayerAuthSetup') > -1) {
-                    var redirect = $('<form>').appendTo(document.body)
-                        .attr({ method: 'POST', action: iframeUrl, target: "_parent" });
-                    redirect.submit();
-                } else {
-                    window.location.href = DOMPurify.sanitize(iframeUrl);
-                }
+        var iframeUrl = readIframeUrl();
+        if (!iframeUrl || iframeUrl === 'about:blank') { return; }
+        if (iframeUrl.indexOf('PayerAuthReturn') > -1) { return; }
+        if (iframeUrl.indexOf(window.location.hostname) > -1) {
+            clearInterval(checkInterval);
+            showProcessing();
+            if (iframeUrl.indexOf('CheckoutServices-PayerAuthSetup') > -1) {
+                var redirect = $('<form>').appendTo(document.body)
+                    .attr({ method: 'POST', action: iframeUrl, target: "_parent" });
```

```
+                redirect.submit();
+            } else {
+                window.location.href = DOMPurify.sanitize(iframeUrl);
             }
-        } catch (e) {
-            // Cross-origin access blocked - this is expected during authentication
         }
     }, 1000);

@@ -60,8 +144,21 @@ function initCardinalPayerAuthModal() {
         clearInterval(checkInterval);
         console.log("Cardinal Authentication timeout, stopping monitoring");
-        modalOverlay.style.display = 'none';
+        if (finished) { return; }
+        finished = true;
+        iframe.hidden = true;
+        if (processing) { processing.hidden = true; }
+        if (timeoutMessage) {
+            timeoutMessage.hidden = false;
+        } else {
+            modalOverlay.style.display = 'none';
+        }
         console.warn("Authentication timed out after 5 minutes");
     }, 300000);
 }
@@ -80,4 +177,4 @@ window.onload = function () {
             stepUpForm.submit();
         }
     }

+};
```

5. Template Changes

deviceDataCollection.isml

Path:
cartridges/int_cybersource_sfra/cartridge/templates/default/payerauthentication/deviceDataCollection.isml

Key changes

```
-<script src="${URLUtils.staticURL('/custom/lib/jquery/jquery-3.7.1.min.js')}"
type="text/javascript"></script>
+<script src="${URLUtils.staticURL('/custom/lib/jquery/jquery-3.7.1.min.js')}"></script>

-<iframe id="cardinal_collection_iframe" ... style="display:none;"></iframe>
+<iframe id="cardinal_collection_iframe" ... hidden></iframe>

-var endpoint= "${dw.system.Site.current.preferences.custom.CsEndpoint.value}";
+// origin derived dynamically from ddcUrl, not from a site preference
 var ddcOrigin = (new URL("${pdict.ddcUrl}")).origin;
```

```
+var redirectSubmitted = false;
+var ddcTimeoutId = null;
+function submitPayerAuthRedirect() {
+    if (redirectSubmitted) { return; }  // guard against message + timeout both firing
+    redirectSubmitted = true;
+    clearTimeout(ddcTimeoutId);
+    document.payerAuthRedirect.submit();
+}

 window.addEventListener("message", function(event) {
     if (event.origin === ddcOrigin) {
-        document.payerAuthRedirect.submit();
+        submitPayerAuthRedirect();
     }
 }, false);

+// 10-second fallback — continue if the DDC iframe never posts a message back
+ddcTimeoutId = setTimeout(function() {
+    console.log("DDC message not received within 10s, continuing...");
+    submitPayerAuthRedirect();
+}, 10000);
```

---

cardinalPayerAuthentication.isml

Path:
`cartridges/int_cybersource_sfra/cartridge/templates/default/cart/cardinalPayerAuthentication.isml`

Added a loading spinner and a timeout message panel inside the step-up iframe container. The iframe itself is now hidden until the challenge loads, so the shopper sees the spinner rather than a blank area while the challenge page loads and again while the return trip (PayerAuthValidate + order placement) runs server-side. A `hidden` timeout div is available to display a user-facing message if the challenge stalls.

```
-                    <iframe id="step-up-iframe" name="step-up-iframe" class="stepup-
iframe"></iframe>
-                    <form id="step-up-form" target="step-up-iframe" method="post"
action="${pdict.stepUpUrl}" class="hidden">
+                    <iscomment>
+                        Shown while the challenge is being loaded, and again once the shopper
has
+                        submitted it - the return trip runs PayerAuthValidate plus order
placement
+                        server side, so the iframe would otherwise sit blank for several
seconds.
+                    </iscomment>
+                    <div id="stepup-processing" class="stepup-status" role="status" aria-
live="polite">
+                        <div class="stepup-spinner-ring"></div>
+                        <p class="stepup-status-
text">${Resource.msg('payment.authentication.processing', 'cybersource', null)}</p>
+                    </div>
+                    <div id="stepup-timeout" class="stepup-status" role="alert" hidden>
```

```
+                           <p class="stepup-status-
text">${Resource.msg('payment.authentication.timeout', 'cybersource', null)}</p>
+                   </div>
+                   <iframe id="step-up-iframe" name="step-up-iframe" class="stepup-iframe"
hidden></iframe>
+                   <form id="step-up-form" target="step-up-iframe" method="post"
action="${pdict.stepUpUrl}" class="hidden">
                            <input type="hidden" name="JWT" value="${pdict.jwtToken}" />
-                           <input type="hidden" name="MD"
value="${pdict.CurrentSession.sessionID}" />
+                           <input type="hidden" name="MD"
value="${pdict.CurrentSession.sessionID}" />
                    </form>
```

Key changes:

- `stepup-processing` div — spinner shown while the challenge iframe loads and during the server-side return trip
- `stepup-timeout` div (initially `hidden`) — available to show `payment.authentication.timeout` if the challenge stalls
- `step-up-iframe` starts `hidden` so the blank iframe is never visible before the challenge page arrives

---

creditCardForm.isml

Path:
`cartridges/int_cybersource_sfra/cartridge/templates/default/checkout/billing/creditCardForm.isml`

Added `payerAuthDDC.js` and `payerAuthCardFields.js` script tags for the SA_FLEX path. jQuery is loaded here (immediately before `flexMicroform.js`) to guarantee DOM order.

```
 <isif condition="${CsSaType && isCartridgeEnabled && CsSaType == 'SA_FLEX'}">
+    <script src="${URLUtils.staticURL('/custom/lib/jquery/jquery-3.7.1.min.js')}"
type="text/javascript"></script>
+    <script src="${URLUtils.staticURL('/custom/payerAuthDDC.js')}"
type="text/javascript"></script>
+    <script src="${URLUtils.staticURL('/custom/payerAuthCardFields.js')}"
type="text/javascript"></script>
     <script src="${URLUtils.staticURL('/custom/flexMicroform.js')}"
type="text/javascript"></script>
     <isinclude url="${URLUtils.url('CYBSecureAcceptance-CreateFlexToken')}"/>
 </isif>
```

---

creditCardContent.isml

Path:
`cartridges/int_cybersource_sfra/cartridge/templates/default/checkout/billing/paymentOptions/creditCardContent.isml`

Renders the `#cyb-payerauth-ddc` host container when `supportsEarlySetup()` returns true. For non-Flex sites also loads jQuery, `payerAuthDDC.js`, and `payerAuthCardFields.js`.

```
+  <isscript>
```

```
+    var cybPayerAuthHelper = require('*/cartridge/scripts/helper/PayerAuthSetupHelper');
+    var cybEarlyPayerAuth =
dw.system.Site.getCurrent().getCustomPreferenceValue('IsCartridgeEnabled')
+        && cybPayerAuthHelper.supportsEarlySetup();
+  </isscript>
+  <isif condition="${cybEarlyPayerAuth}">
+    <div id="cyb-payerauth-ddc"
+         data-setup-url="${URLUtils.https('CheckoutServices-PayerAuthSetupData')}"
+         data-save-device-data-url="${URLUtils.https('CheckoutServices-SaveDeviceData')}"
+         data-clear-url="${URLUtils.https('CheckoutServices-ClearPayerAuthSetup')}"
+         data-capture-context-url="${URLUtils.https('CYBSecureAcceptance-FlexCaptureContext')}"
+         data-payer-auth-card-types="${cybPayerAuthCardTypes}"
+         hidden></div>
+    <isif condition="${CsSaType != 'SA_FLEX'}">
+      <script src="${URLUtils.staticURL('/custom/lib/jquery/jquery-3.7.1.min.js')}"
type="text/javascript"></script>
+      <script src="${URLUtils.staticURL('/custom/payerAuthDDC.js')}"
type="text/javascript"></script>
+      <script src="${URLUtils.staticURL('/custom/payerAuthCardFields.js')}"
type="text/javascript"></script>
+    </isif>
+  </isif>
```

---

storedPaymentInstruments.isml

Path:
`cartridges/int_cybersource_sfra/cartridge/templates/default/checkout/billing/storedPaymentInstruments.isml`

Added `data-card-type` attribute so `payerAuthCardFields.js` can determine payer auth eligibility without parsing display text.

```
-    <div class="row saved-payment-instrument ${loopSate.first ? 'selected-payment' : ''}" data-
uuid="${paymentInstrument.UUID}">
+    <div class="row saved-payment-instrument ${loopSate.first ? 'selected-payment' : ''}"
+         data-uuid="${paymentInstrument.UUID}"
+         data-card-type="${paymentInstrument.creditCardType}">
```

---

6. Styling Changes

cyber-source.scss

Path: `cartridges/int_cybersource_sfra/cartridge/client/default/scss/cyber-source.scss`

Added styles for the 3DS challenge step-up spinner and status overlay.

```
+.stepup-status {
+    width: 100%; height: 400px;
+    flex-direction: column; justify-content: center; align-items: center;
+    display: flex;
+    &[hidden] { display: none; }
+}
+
+.stepup-spinner-ring {
+    width: 40px; height: 40px;
```

```
+    border: 4px solid #e0e0e0; border-top-color: #333;
+    border-radius: 50%;
+    animation: stepup-spin 0.8s linear infinite;
+}
+
+.stepup-status-text {
+    margin: 16px 0 0; padding: 0 16px;
+    text-align: center; color: #333;
+}
+
+@keyframes stepup-spin { to { transform: rotate(360deg); } }
+
+@media (prefers-reduced-motion: reduce) {
+    .stepup-spinner-ring { animation-duration: 2.4s; }
+}
```

---

global.scss

Path: `cartridges/int_cybersource_sfra/cartridge/client/default/scss/global.scss`

Overrides SFRA spinner dot colour to light blue, scoped to `.veil .spinner` to outrank the base rules without relying on import order.

```
+$cyb-spinner-color: #add8e6;
+
+.veil .spinner {
+    .dot1, .dot2 {
+        background-color: $cyb-spinner-color;
+    }
+}
```

---

7. Build & Dependency Changes

webpack.config.js

Path: `webpack.config.js`

Replaced `sgmf-scripts.createJsPath()` / `createScssPath()` with a native `fs`-based directory walker.

Root cause: On Windows, `sgmf-scripts` builds glob patterns using `path.join()`, which produces backslash-separated strings. The `glob` library treats a backslash as an escape character, so the pattern matched nothing and webpack silently emitted no JS or CSS.

Note: this file change can be ignored if code compilation is successful with existing wepack.config.js

```
-var sgmfScripts = require('sgmf-scripts');
+var fs = require('fs');
+
+var CLIENT_DIR = path.resolve('./cartridges/int_cybersource_sfra/cartridge/client');
+
+function walk(dir) {
+    if (!fs.existsSync(dir)) { return []; }
+    return fs.readdirSync(dir, { withFileTypes: true }).reduce(function (files, entry) {
```

```
+        var full = path.join(dir, entry.name);
+        return files.concat(entry.isDirectory() ? walk(full) : [full]);
+    }, []);
+}
+
+function relativeName(filePath) {
+    return path.relative(CLIENT_DIR, filePath).split(path.sep).join('/');  // always forward
slashes
+}
+
+function createJsPath() {
+    return walk(CLIENT_DIR).reduce(function (result, filePath) {
+        var name = relativeName(filePath);
+        if (/(^|\/)js\/[^/]+\.js$/.test(name)) { result[name.slice(0, -3)] = filePath; }
+        return result;
+    }, {});
+}
+
+function createScssPath() {
+    return walk(CLIENT_DIR).reduce(function (result, filePath) {
+        var name = relativeName(filePath);
+        if (/(^|\/)scss\/.*\.scss$/.test(name) && path.basename(filePath).indexOf('_') !== 0) {
+            result[name.slice(0, -5).replace('scss', 'css')] = filePath;
+        }
+        return result;
+    }, {});
+}

 module.exports = [{
-    entry: sgmfScripts.createJsPath(),
+    entry: createJsPath(),
 ...
-    entry: sgmfScripts.createScssPath(),
+    entry: createScssPath(),
```

package.json

```
-  "version": "26.1.1",
+  "version": "26.1.2",

-    "dompurify": "^3.2.6",
+    "dompurify": "^3.4.13",        ← security patch

   "overrides": {
-    "brace-expansion": "5.0.8",
+    "brace-expansion": "5.0.9",
+    "nanoid": "^5.0.0",            ← new override
+    "fast-uri": "^3.1.5",          ← new override
+    "js-yaml": "^4.3.1"            ← new override
   }
```

8. Version & Configuration Changes

CybersourceConstants.js

```
-CybersourceConstants.APPLICATION_VERSION = '26.1.1';
+CybersourceConstants.APPLICATION_VERSION = '26.1.2';
```

README.md

```
-* **Version:** 26.1.1
+* **Version:** 26.1.2
```

---

cybersource.properties

```
+# Payer Authentication setup (device data collection at card entry)
+error.message.payerauthsetup.fail = Something went wrong while verifying your card. Please check
your card details and try again. If this issue continues, please contact customer support.
+
+# Payer Authentication step-up modal (3DS challenge)
+payment.authentication.title = Payment Authentication
+payment.authentication.processing = Please wait whilst we process your payment. Your card issuer
may ask for additional details.
+payment.authentication.timeout = Your payment authentication timed out. Please return to
checkout and try again.
```

Note:

- Refer to payment.authentication.processing in cybersource.properties file to update the custom message that needs to be displayed on payer auth page.
- Refer to payerAuthDDC.js to set timeout to DDC and button hold.